KRISTU JAYANTI
(DEEMED TO BE UNIVERSITY)
Under Section 3 of UGC Act 1956
A CMI INSTITUTION | BENGALURU | INDIA

Project Report on

"WI-FI SECURITY PROFILER"

A CYBERSECURITY CASE STUDY

Submitted in partial fulfilment of the practical requirements for
PYTHON FOR CYBER SECURITY PRACTICAL
Master of Science in Cyber Security

by

CHRIS MATHEW AJE
Register Number: 26CYBR10

Under the Guidance of
DR. RAMYA B

Sem I
Kristu Jayanti (Deemed to be University)

TABLE OF CONTENTS

Chapter. No.	Contents	Page No.
1	INTRODUCTION	1–8
1.1	Introduction to Cybersecurity and Wireless Network Vulnerabilities	1
1.2	Wi-Fi Security Standards and Encryption Protocols	2
1.3	Common Wi-Fi Vulnerabilities and Security Threat Vectors	4
1.4	Role of Python in Cybersecurity and Network Auditing	5
1.5	Overview and Methodology of Proposed Project	7
1.6	Importance of Wi-Fi Security Profiling and Organisation of Project Report	21
2	OBJECTIVES OF THE PROJECT	9–11
2.1	Introduction	9
2.2	Primary Objective	9
2.3	Real-Time Wireless Telemetry Extraction and Hardware Detection	9
2.4	Captive Portal and Internal Web Login Identification	21
2.5	Rule-Based Vulnerability Pattern Engine and Risk Scoring	10
2.6	Multi-User Security, Web Dashboard and Report Exporting	11
3	SCOPE OF THE PROJECT	12–15
3.1	Introduction	12
3.2	Wireless Network Parameters and Security Attributes Covered	12
3.3	Hardware Querying vs. Manual Audit Capabilities	13
3.4	Scoring Model, Risk Classification and Mitigation Guidelines	13

Chapter. No.	Contents	Page No.
3.5	Tools, Technologies and Target Execution Environments	14
3.6	Scope Limitations and Educational Application	15
4	SOFTWARE SPECIFICATIONS	16–22
4.1	Introduction	16
4.2	Programming Language and Development Environment	16
4.3	Python Libraries Used in the Project	16
4.4	Python Programming Concepts	18
4.5	Software Requirements and Testing Environment	20
4.6	Role of Software Components in Project Execution	21
5	WORKFLOW OF THE PROJECT	23–26
5.1	Introduction	23
5.2	Overall Project Workflow	23
5.3	Phase 1 – Telemetry Acquisition and Environmental Detection	24
5.4	Phase 2 – Vulnerability Analysis and Quantitative Risk Scoring	25
5.5	Phase 3 – Web Visualization, User Session Management and Audit Reporting	26
6	MODULES OF THE PROJECT	27–31
6.1	Introduction	27
6.2	Module 1 – Wi-Fi Detector Module (wifi_detector.py)	27
6.3	Module 2 – Risk Evaluator Engine (risk_evaluator.py)	28
6.4	Module 3 – Authentication & Security Module (auth.py)	28
6.5	Module 4 – Web Application & Presentation Controller (app.py)	29

Chapter. No.	Contents	Page No.
6.6	Module 5 – Terminal Command Line Interface (main.py)	29
6.7	Integration and Overall Outcome of the Modules	30
7	SOURCE CODE	32–41
7.1	Introduction	32
7.2	Core Telemetry and Captive Portal Detection Implementation	32
7.3	Vulnerability Pattern Matching and Risk Scoring Engine	34
7.4	Cryptographic User Authentication and Data Persistence	37
7.5	Flask Application Controller and Export Handlers	38
7.6	Automated Test Suite and Verification	40
8	FEATURES OF THE PROJECT	42–43
8.1	Introduction	42
8.2	Automated Hardware Telemetry and Live Network Detection	42
8.3	Dual-Stage Captive Portal / Web Splash Page Detection	42
8.4	Transparent 0-100 Risk Scoring and Actionable Hardening Recommendations	42
8.5	Secure Multi-User Portal with SHA-256 Authentication	43
8.6	Interactive Cybersecurity Dashboard with Manual Auditing & Multi-Format Exports	43
8.7	Dual Operational Modes (Web UI and CLI) and Comprehensive Test Suite	43
9	FUTURE ENHANCEMENTS	44–45
9.1	Introduction	44
9.2	Cross-Platform Wireless Telemetry Extraction	44

Chapter. No.	Contents	Page No.
9.3	Machine Learning-Based Rogue AP and Evil Twin Detection	44
9.4	Real-Time Continuous Background Monitoring and Audio/Desktop Alerts	44
9.5	Packet Capture (PCAP) Analysis and Deauthentication Frame Detection	45
9.6	Automated PDF Audit Certificate Generation	45
10	CONCLUSION	46–47
10.1	Introduction	46
10.2	Summary of Project Accomplishments	46
10.3	Role of Python in Wi-Fi Security Auditing	46
10.4	Results and Learning Outcomes	47
10.5	Limitations and Overall Project Evaluation	47
10.6	Overall Project Significance and Conclusion	47
	REFERENCES	48

LIST OF TABLES

Table No.	Title	Page No.
Table 1.1	Wi-Fi Security Protocols and Encryption Standards	03
Table 1.2	Python Standard and Third-Party Libraries Used in Wi-Fi Security Profiler	06
Table 3.1	Functional Areas Covered and Architectural Boundaries	14
Table 4.1	Python Libraries, Modules and Functional Purpose	17
Table 4.2	Core Python Programming Concepts and Their System Application	19
Table 4.3	Minimum Software, Hardware and Runtime Specifications	20
Table 5.1	Multi-Phase Processing Workflow of Wi-Fi Security Profiler	23
Table 6.1	Evaluated Wi-Fi Security Audit Case Studies	30

LIST OF ABBREVIATIONS

AES – Advanced Encryption Standard. AP – Access Point. API – Application Programming Interface. BSSID – Basic Service Set Identifier. CCMP – Counter Mode Cipher Block Chaining Message Authentication Code Protocol. CLI – Command Line Interface. CPU – Central Processing Unit. CRC – Cyclic Redundancy Check. CSV – Comma-Separated Values.

DNS – Domain Name System. EAP – Extensible Authentication Protocol. EAPOL – Extensible Authentication Protocol over LAN. GCMP – Galois/Counter Mode Protocol. GUI – Graphical User Interface. HTTP – Hypertext Transfer Protocol. HTTPS – Hypertext Transfer Protocol Secure. IDE – Integrated Development Environment. IEEE – Institute of Electrical and Electronics Engineers.

IoC – Indicator of Compromise. IP – Internet Protocol. IV – Initialization Vector. JSON – JavaScript Object Notation. MAC – Media Access Control. MIC – Message Integrity Code. MITM – Man-in-the-Middle. NIC – Network Interface Controller. OS – Operating System.

PBKDF2 – Password-Based Key Derivation Function 2. PCAP – Packet Capture. PID – Process Identifier. PIN – Personal Identification Number. PMK – Pairwise Master Key. PMKID – Pairwise Master Key Identifier. PTK – Pairwise Transient Key. RAM – Random Access Memory. RC4 – Rivest Cipher 4.

Regex – Regular Expression. REST – Representational State Transfer. RFC – Request for Comments. SAE – Simultaneous Authentication of Equals. SHA – Secure Hash Algorithm. SSID – Service Set Identifier. TCP – Transmission Control Protocol. TKIP – Temporal Key Integrity Protocol. UDP – User Datagram Protocol.

UI – User Interface. URL – Uniform Resource Locator. VM – Virtual Machine. WEP – Wired Equivalent Privacy. Wi-Fi – Wireless Fidelity. WLAN – Wireless Local Area Network. WPA – Wi-Fi Protected Access. WPA2 – Wi-Fi Protected Access 2. WPA3 – Wi-Fi Protected Access 3.

WPS – Wi-Fi Protected Setup.

These abbreviations are included to make the report easier to read and understand. They represent commonly used terms related to cybersecurity, IEEE 802.11 wireless networking, cryptographic algorithms, Python programming, and computer systems. They are used in the context of network telemetry examination, captive portal redirection, rule-based vulnerability identification, and web application auditing throughout the project.

The abbreviations provide a quick reference to technical terminology used across the report. Some terms are included to explain general wireless cybersecurity concepts and protocol standards and may not be directly implemented in the current software release.

CHAPTER 1 – INTRODUCTION

1.1 Introduction to Cybersecurity and Wireless Network Vulnerabilities

Cybersecurity is the discipline dedicated to defending digital infrastructure, communications channels, application logic, and confidential data assets against unauthorised access, malicious interception, disruption, and exploitation. In modern computing topologies, wireless local area networks (WLANs) operating under the IEEE 802.11 international standard have largely supplanted conventional wired Ethernet for endpoint connectivity across academic institutions, corporate enterprises, public transportation hubs, and private households. While wireless connectivity affords exceptional mobility and administrative flexibility, it introduces fundamental architectural vulnerabilities that do not exist within physical, cable-bounded networks.

In a conventional wired local network, physical perimeter security controls—such as building access gates, locked server rooms, and patch panel enclosures—prevent unauthorized physical taps onto network cables. Conversely, wireless communications rely on omnidirectional radio frequency (RF) signal propagation across the unguided 2.4 GHz, 5 GHz, and 6 GHz spectrum bands. Any transceiver or wireless network interface controller (NIC) situated within physical range of an access point can capture raw radio frames transmitted over the air. Unless robust cryptographic encapsulation is enforced at the data link layer, an adversary requires no physical intrusion to inspect network payloads, hijack sessions, or spoof operational parameters.

The fundamental security objectives of confidentiality, integrity, availability, and non-repudiation face distinct threats in wireless environments. Passive eavesdropping allows attackers to capture cleartext application data, unencrypted HTTP traffic, and user credentials. Active wireless attacks—such as rogue access points, Evil Twin spoofing, deauthentication flooding, and Man-in-the-Middle (MITM) frame manipulation—allow malicious actors to position themselves transparently between legitimate clients and gateway routers. Consequently, comprehensive wireless security evaluation requires rigorous automated auditing of interface configurations, active encryption protocols, key exchange handshakes, and captive portal mechanisms.

Despite significant advances in wireless security engineering, legacy standards and insecure configuration profiles remain widely deployed. Many organizations maintain backward compatibility with obsolete encryption algorithms or provide open guest networks that expose end users to significant risks. Automated security profiling provides a systematic methodology for interrogating active wireless

adapters, evaluating encryption parameters, detecting deceptive captive portals, and delivering actionable hardening guidance to mitigate potential cyber threats before sensitive data is exposed.

1.2 Wi-Fi Security Standards and Encryption Protocols

The evolution of IEEE 802.11 security standards reflects an ongoing cryptographic competition between protocol designers and security researchers. The original standard, Wired Equivalent Privacy (WEP), was ratified in 1997 with the ambitious objective of delivering data confidentiality comparable to wired networks. WEP employed the Rivest Cipher 4 (RC4) symmetric stream cipher, utilizing either 40-bit or 104-bit static keys combined with a 24-bit Initialization Vector (IV) to form 64-bit or 128-bit key seeds. However, the cryptographic implementation was fundamentally flawed due to the limited 24-bit IV size. Under moderate network traffic, identical IV values repeat within hours due to the birthday paradox. In 2001, cryptanalysts Fluhrer, Mantin, and Shamir (FMS) demonstrated that weak key scheduling in RC4 allows an attacker to recover the complete secret key by collecting fewer than one million data frames, rendering WEP obsolete.

To address the immediate vulnerabilities of WEP without requiring hardware replacements across existing client adapters, the Wi-Fi Alliance introduced Wi-Fi Protected Access (WPA) in 2003 as an interim specification. WPA incorporated the Temporal Key Integrity Protocol (TKIP) and the Michael Message Integrity Code (MIC). TKIP extended the IV space to 48 bits, implemented a per-packet key mixing function, and incorporated sequence counters to defend against replay attacks. Despite these improvements, TKIP remained tethered to the underlying RC4 cipher architecture. Subsequent research, including the Beck-Tews attack and the Ohigashi-Morii attack, demonstrated practical keystream recovery and packet injection vulnerabilities, prompting standards bodies to deprecate TKIP entirely.

In 2004, the IEEE ratified the 802.11i amendment, commercialized as Wi-Fi Protected Access 2 (WPA2). WPA2 superseded RC4 by mandating the Advanced Encryption Standard (AES) operating within the Counter Mode Cipher Block Chaining Message Authentication Code Protocol (CCMP). AES-CCMP provides authenticated encryption, guaranteeing both confidentiality via CTR mode and data origin authenticity via CBC-MAC. While WPA2-Enterprise leverages IEEE 802.1X Extensible Authentication Protocol (EAP) frameworks with RADIUS servers, the pervasive WPA2-Personal configuration utilizes a Pre-Shared Key (PSK). WPA2-Personal derives a 256-bit Pairwise Master Key (PMK) using the PBKDF2 key derivation function over 4,096 iterations. However, WPA2-Personal remains susceptible to offline dictionary attacks if an attacker intercepts the 4-way EAPOL handshake or captures the Pairwise Master Key Identifier (PMKID) from the first message.

To eliminate handshake interception vulnerabilities, the Wi-Fi Alliance announced Wi-Fi Protected Access 3 (WPA3) in 2018. WPA3 replaces PSK negotiation with Simultaneous Authentication of Equals (SAE), a cryptographic handshake based on the Dragonfly key exchange protocol. SAE utilizes discrete logarithm zero-knowledge proofs to authenticate endpoints without transmitting verifiable password tokens over the air, delivering forward secrecy and resilience against offline dictionary attacks. Additionally, WPA3 mandates Protected Management Frames (PMF / IEEE 802.11w) to prevent unauthorized deauthentication frame spoofing.

Alongside encryption protocols, auxiliary connection mechanisms such as Wi-Fi Protected Setup (WPS) were developed to simplify home router pairing. However, WPS introduced severe security vulnerabilities. The WPS external registrar PIN authentication protocol splits an 8-digit PIN into two distinct halves (4 digits and 3 digits plus a checksum), independently validated by the access point. This design flaw reduces the computational brute-force complexity from 100,000,000 possibilities to a mere 11,000 guesses, allowing automated attack utilities such as Reaver and Pixie Dust to compromise the network key within minutes if WPS is left active.

The technical characteristics, release timelines, cryptographic primitives, and known vulnerability profiles of these wireless standards are summarized in Table 1.1.

Table 1.1: Wi-Fi Security Protocols and Encryption Standards

Security Protocol	Release Year	Underlying Cipher Suite	Key Exchange Mechanism	Known Vulnerabilities & Attack Vectors
WEP (64/128)	1997	RC4 Stream Cipher	Static Pre-Shared Key (24-bit IV)	IV collision, FMS attack, complete key recovery within minutes
WPA	2003	TKIP / Michael MIC	4-Way Handshake (PSK / 802.1X)	Beck-Tews packet injection, Michael MIC weakness, downgrade attacks
WPA2-Personal	2004	AES-CCMP (128-bit)	4-Way Handshake (PSK)	Offline 4-way dictionary attacks, PMKID capture, KRACK vulnerability
WPA2-Enterprise	2004	AES-CCMP (128-bit)	EAP-TLS / PEAP / EAP-TTLS	Misconfigured RADIUS trust, Rogue AP evil twin certificate spoofing

Security Protocol	Release Year	Underlying Cipher Suite	Key Exchange Mechanism	Known Vulnerabilities & Attack Vectors
WPA3-Personal	2018	AES-CCMP / GCMP	SAE (Dragonfly Handshake)	Dragonblood side-channel timing attacks (mitigated in errata), PMF mandated
Open / Portal	N/A	None (Plaintext)	No Key Exchange / Web Redirect	Cleartext interception, unencrypted credential sniffing, MITM injection

1.3 Common Wi-Fi Vulnerabilities and Security Threat Vectors

Wireless network deployments present an extensive attack surface characterized by diverse vulnerability patterns. The most elementary yet pervasive hazard is the unencrypted open network. In an open wireless architecture, data frames are transmitted in cleartext without link-layer cryptographic transformation. Any observer operating a wireless adapter in promiscuous or monitor mode can intercept complete application payloads, HTTP session identifiers, cleartext authentication cookies, and DNS lookups. Furthermore, open networks allow attackers to execute Address Resolution Protocol (ARP) cache poisoning and rogue DHCP server injections to redirect victim traffic through adversary-controlled gateways.

Captive portals represent a prominent threat vector commonly encountered on public, institutional, and hospitality Wi-Fi networks. In a typical captive portal workflow, the access point associates with the client without 802.11 encryption, establishing an unencrypted wireless link. When the client attempts to reach the internet, the gateway intercepts initial TCP/HTTP traffic on port 80 and issues an HTTP 302 redirect to an internal splash or web login page. Users are prompted to enter credentials, access codes, or personal data. Because the underlying wireless link remains completely unencrypted, an adversary can sniff credentials directly from over-the-air HTTP requests or execute SSL stripping attacks to downgrade secured HTTPS connections to plaintext.

Rogue access points and Evil Twin attacks exploit the standard 802.11 client roaming behavior. Operating systems automatically connect to known wireless network names (SSIDs) broadcast by nearby access points, frequently prioritizing the access point broadcasting the highest Received Signal Strength Indicator (RSSI). An adversary can configure a rogue Wi-Fi transceiver to broadcast identical SSID parameters and BSSID characteristics as a legitimate organization network. By transmitting spoofed deauthentication frames to disconnect the victim from the legitimate AP, the attacker forces the

client device to associate with the malicious clone, facilitating complete packet inspection and credential harvesting.

Legacy encryption algorithms—specifically WEP and WPA-TKIP—represent lingering threats caused by administrative oversight or backward compatibility configurations. Even when an access point supports modern WPA2-AES, enabling mixed WPA/WPA2 or TKIP compatibility compromises the entire network. Attackers can execute cryptographic downgrade attacks, injecting packets or exploiting weak initialization vectors to decrypt network traffic. Finally, inadequate passphrase entropy in WPA2-Personal deployments permits high-speed dictionary and brute-force computation using specialized GPU clusters once an EAPOL handshake is captured.

1.4 Role of Python in Cybersecurity and Network Auditing

Python has firmly established itself as the programming language of choice for cybersecurity practitioners, penetration testers, security analysts, and forensic researchers. Python combines expressive, high-level syntax with rapid prototyping capabilities and an expansive standard library. In network auditing, Python excels at automating repetitive administrative tasks, orchestrating operating system commands, parsing complex multiline telemetry streams, executing cryptographic validations, and presenting analytical findings through structured interfaces.

A significant strength of Python in systems-level network security is its ability to interface directly with host operating system utilities via the standard `subprocess` module. Rather than requiring developers to write complex, platform-dependent C extensions or install third-party kernel drivers, Python can invoke native administrative command-line tools—such as Windows `netsh wlan`—and capture the resulting text streams for programmatic processing. Coupled with Python's compiled regular expression engine (`re`), multi-line terminal outputs containing interface states, SSID names, MAC addresses, encryption suites, and signal percentages can be parsed into normalized data dictionaries with minimal latency.

Python's standard library provides robust security primitives out of the box. The `hashlib` module offers cryptographic hash functions including SHA-256 and SHA-512, enabling deterministic credential hashing and integrity verification without external dependencies. Built-in serialization modules such as `json` and `csv` facilitate rapid data interchange and persistent audit trail recording. For network probing, Python's `urllib.request` enables lightweight HTTP request dispatching with customizable headers and redirect handling, ideal for probing captive portal detection endpoints.

Beyond systems scripting, Python supports rapid full-stack web application development through frameworks such as Flask. Flask provides a lightweight WSGI framework featuring URL routing, session cookie signing, template rendering via Jinja2, and RESTful API creation. This flexibility allows security tools to offer both an interactive, dark-themed browser interface for end-user engagement and a lightweight terminal CLI for automated headless server execution.

The core Python standard library modules and third-party dependencies utilized in the architecture of the Wi-Fi Security Profiler are detailed in Table 1.2.

Table 1.2: Python Standard and Third-Party Libraries Used in Wi-Fi Security Profiler

Library / Module	Library Type	Version / Origin	Primary Functional Role in Profiler Architecture
Flask	Third-Party Framework	Version 3.1.3	WSGI web routing, session state management, template dispatching, REST API
subprocess	Python Standard Library	Built-in (CPython)	Executing Windows netsh wlan commands and streaming standard output bytes
re	Python Standard Library	Built-in (CPython)	Compiling regular expressions for parsing multiline wireless interface telemetry
hashlib	Python Standard Library	Built-in (CPython)	Computing deterministic SHA-256 digests for secure multi-user credential storage
json	Python Standard Library	Built-in (CPython)	Serializing evaluated Wi-Fi audit profiles and user database records to disk
csv	Python Standard Library	Built-in (CPython)	Formatting and streaming tabular security audit reports for analyst download
urllib.request	Python Standard Library	Built-in (CPython)	Executing HTTP connectivity probe requests to detect captive portal splash redirects

Library / Module	Library Type	Version / Origin	Primary Functional Role in Profiler Architecture
unittest	Python Standard Library	Built-in (CPython)	Automated testing framework verifying 29 functional and security test cases

1.5 Overview and Methodology of Proposed Project

The proposed project, entitled 'Wi-Fi Security Profiler' (Check the WifiSec), is an automated, modular wireless vulnerability assessment and risk classification system developed entirely in Python. The system provides immediate, transparent insight into the security posture of active and nearby wireless networks, empowering users and system administrators to identify insecure communication channels before confidential data is transmitted over vulnerable radio links.

The operational methodology of the Wi-Fi Security Profiler encompasses three primary functional stages: Telemetry Acquisition, Vulnerability & Risk Evaluation, and Presentation/Reporting. In the Telemetry Acquisition phase, the tool interfaces directly with the host operating system's wireless subsystem via Windows `netsh wlan` commands. It dynamically extracts critical operational parameters from the active wireless network adapter, including the Service Set Identifier (SSID), Basic Service Set Identifier (BSSID), 802.11 radio standard (e.g., 802.11ax, 802.11ac, 802.11n), signal quality percentage, 802.11 authentication standard (e.g., Open, WEP, WPA2-Personal, WPA3), and encryption cipher (e.g., None, TKIP, CCMP/AES).

Simultaneously, the tool executes dual-stage captive portal detection. It dispatches non-intrusive HTTP probe requests to standard connectivity endpoints (`http://captive.apple.com/hotspot-detect.html` and `http://www.msftconnecttest.com/connecttest.txt`), inspecting HTTP response codes and payload contents for redirection patterns indicative of internal web login pages. This network probe is supplemented by lexical heuristic pattern matching against the SSID to detect public hotspot identifiers (e.g., 'Guest', 'Airport', 'Free', 'Hotel').

In the Vulnerability & Risk Evaluation phase, the extracted telemetry dictionary is processed by a rule-based risk evaluation engine (`risk_evaluator.py`). The engine evaluates the profile against six predefined vulnerability patterns: Unencrypted Open Network (`OPEN_NETWORK`), Captive Portal / Web Splash Page (`CAPTIVE_PORTAL`), Deprecated WEP Encryption (`WEP_DEPRECATED`), Legacy TKIP Cipher (`TKIP_LEGACY`), WPS PIN Vulnerability (`WPS_ENABLED`), and Weak Passphrase Entropy (`WEAK_PASSWORD`). Each identified pattern introduces weighted

mathematical penalties, which are combined with a baseline security calculation and clamped to a normalized 0–100 scale. The composite score determines the network's risk tier: `CRITICAL` (80–100), `HIGH` (60–79), `MEDIUM` (40–59), or `LOW` (0–39).

In the Presentation and Reporting phase, the system provides dual interfaces. The primary interface is a cybersecurity web dashboard built with Flask, styled in a dark monochrome aesthetic with neon risk badges, summary cards, and responsive controls. Users can trigger live hardware auto-detection, scan visible nearby access points, input custom parameters via an interactive manual audit modal, and download timestamped audit logs in CSV or JSON formats. An auxiliary terminal CLI (`main.py`) provides equivalent functionality for console environments.

1.6 Importance of Wi-Fi Security Profiling and Organisation of Project Report

Automated wireless security profiling is an indispensable component of defensive cybersecurity engineering. End users frequently connect their computing devices to available wireless access points based solely on signal strength or connection convenience, without understanding the underlying cryptographic safeguards or exposure risks. By translating complex 802.11 parameters into intuitive risk scores and providing clear, actionable remediation steps (such as enabling a virtual private network, disabling WPS, or upgrading to WPA3-AES), the Wi-Fi Security Profiler bridges the gap between intricate wireless protocol mechanics and practical security hygiene.

The remainder of this project report is structured systematically into the following chapters:

Chapter 2 defines the precise technical and operational objectives of the project. Chapter 3 delineates the functional scope, parameter coverage, architectural boundaries, and defensive limitations. Chapter 4 provides the software and hardware specifications, detailing the development environment, library dependencies, and core programming paradigms. Chapter 5 details the end-to-end multi-phase workflow pipeline. Chapter 6 breaks down the internal modular architecture and case study evaluations. Chapter 7 presents and analyzes the core Python source code implementations. Chapter 8 highlights the key capabilities and security features. Chapter 9 outlines future research directions and engineering enhancements. Finally, Chapter 10 presents the project conclusions, learning outcomes, and technical evaluation, followed by academic references.

CHAPTER 2 - OBJECTIVES OF THE PROJECT

2.1 Introduction

The overarching goal of the Wi-Fi Security Profiler project is to develop an automated, reliable, and secure software application in Python capable of interrogating host wireless interfaces, evaluating wireless network security parameters, identifying vulnerability signatures, and classifying overall risk levels. This chapter articulates the primary and secondary objectives established to guide the architecture, development, and testing of the system.

Developing an automated wireless security tool requires solving multiple engineering challenges: interacting cleanly with operating system networking commands without external driver dependencies, parsing diverse terminal outputs into structured objects, accurately detecting captive portal redirections without interfering with active user connections, designing an objective mathematical risk formula, and securing the resulting application with cryptographic user authentication. The objectives defined herein establish the functional benchmarks against which project success is evaluated.

2.2 Primary Objective

The primary objective of this project is to design, implement, and validate a standalone, modular Wi-Fi Security Profiling and Risk Assessment system in Python that automates the extraction of wireless interface telemetry, detects captive portal splash pages, identifies specific vulnerability patterns, assigns quantitative risk scores on a 0–100 scale, and provides contextual remediation recommendations through an intuitive web interface and a terminal CLI.

To satisfy this primary objective, the application must operate without requiring administrative root/kernel modifications, relying entirely on native operating system tools and Python standard library capabilities. The software must deliver consistent, deterministic risk assessments across diverse network environments—from unencrypted campus guest portals to enterprise-grade WPA3 networks—ensuring that users receive immediate, comprehensible feedback regarding their communications security posture.

2.3 Real-Time Wireless Telemetry Extraction and Hardware Detection

A foundational objective is the implementation of a robust, hardware-independent telemetry extraction mechanism within `wifi_detector.py`. The system must automatically query the active wireless interface card on the host machine using Python's `subprocess` module, invoking native Windows `netsh wlan` administrative utilities.

The telemetry extraction module must successfully capture and normalize the following connection attributes: the active Service Set Identifier (SSID), the hardware MAC address of the serving access point (BSSID), the current interface operational state, the 802.11 radio physical standard (such as 802.11ax, 802.11ac, or 802.11n), the physical frequency band, the radio signal reception quality expressed as an exact percentage, the 802.11 link-layer authentication protocol, and the active encryption cipher suite.

Furthermore, the hardware module must provide secondary scanning capabilities, allowing the application to discover and display all visible nearby wireless networks, capturing their respective SSIDs, signal strengths, authentication types, and ciphers to facilitate comparative environmental risk assessments.

2.4 Captive Portal and Internal Web Login Identification

A critical objective is developing a dependable, non-intrusive captive portal identification mechanism. Public and institutional Wi-Fi networks frequently broadcast open 802.11 associations but intercept subsequent web requests, redirecting users to an internal HTTP web splash page or authentication portal where credentials must be entered.

The objective is to implement a dual-stage detection architecture. Stage one executes an active HTTP network probe using Python's `urllib.request` to query standardized network connectivity test endpoints (`http://captive.apple.com/hotspot-detect.html` and `http://www.msftconnecttest.com/connecttest.txt`). By inspecting HTTP status codes and checking response bodies for expected validation tokens (such as 'Success'), the module reliably identifies gateway redirects. Stage two implements a fallback lexical heuristic engine that analyzes SSID names against known public network keywords ('Guest', 'Airport', 'Hotel', 'Campus', 'Free') to ensure accurate detection even when client internet access is completely blocked.

2.5 Rule-Based Vulnerability Pattern Engine and Risk Scoring

Another core objective is the engineering of an objective, rule-based risk evaluation engine in `risk_evaluator.py`. The engine must systematically evaluate extracted wireless profiles against six defined vulnerability signatures: unencrypted open associations, captive portal credential interception risks, deprecated WEP algorithms, legacy TKIP ciphers, active WPS PIN pairing vulnerabilities, and weak passphrase entropy.

The objective requires formulating a transparent mathematical scoring model that calculates a composite risk score spanning 0 to 100. The scoring logic must apply weighted penalty deductions based on

vulnerability severity, adjust for password complexity ratings, and enforce strict upper and lower numerical bounds. Furthermore, the engine must map calculated numerical scores to discrete risk categories (`CRITICAL`, `HIGH`, `MEDIUM`, `LOW`) and dynamically generate context-specific security hardening recommendations for each audited profile.

2.6 Multi-User Security, Web Dashboard and Report Exporting

The final technical objective encompasses application security, presentation delivery, and compliance reporting. The system must implement a multi-user authentication module (`auth.py`) that enforces input validation, prevents case-insensitive username collisions, and hashes user credentials using the cryptographic SHA-256 algorithm before storing them in `users.json`.

The presentation layer must deliver a responsive, dark-themed cybersecurity dashboard via Flask (`app.py`), featuring glassmorphism cards, real-time connection telemetry badges, summary statistical metrics, and an interactive manual audit modal that permits security analysts to test custom network parameters. Finally, the application must provide robust report generation, allowing users to export evaluated audit records in standardized CSV and JSON formats for documentation and compliance auditing.

CHAPTER 3 – SCOPE OF THE PROJECT

3.1 Introduction

Defining the scope of a cybersecurity tool is essential for establishing functional boundaries, architectural assumptions, operational constraints, and intended usage contexts. The Wi-Fi Security Profiler is conceived as a lightweight, automated, defensive auditing solution designed for personal security awareness, academic network laboratory evaluations, and administrative compliance verification.

This chapter delineates the wireless parameters and encryption standards evaluated by the system, compares automated hardware detection against manual hypothetical auditing, formalizes the risk scoring boundaries, details the software execution environments, and articulates explicit technical and ethical limitations.

3.2 Wireless Network Parameters and Security Attributes Covered

The operational scope of the Wi-Fi Security Profiler encompasses the comprehensive interrogation of data link layer and physical layer attributes defined under the IEEE 802.11 standards family. The system extracts, normalizes, and evaluates the following key parameters:

1. Service Set Identifier (SSID): The alphanumeric network name broadcast by access points or assigned to hidden networks, utilized for network identification and heuristic captive portal pattern matching.

2. Basic Service Set Identifier (BSSID): The 48-bit MAC address corresponding to the radio transceiver of the connected access point, crucial for distinguishing between individual APs in multi-AP enterprise meshes and detecting potential Evil Twin clones.

3. 802.11 Authentication Protocols: Formal evaluation of Open System authentication, legacy Shared Key authentication (WEP), WPA-Personal (PSK), WPA2-Personal (PSK), WPA2-Enterprise (802.1X), WPA3-Personal (SAE), and WPA3-Enterprise.

4. Encryption Cipher Suites: Explicit inspection of link-layer ciphers, including None (cleartext), WEP-40 (64-bit), WEP-104 (128-bit), TKIP (RC4-based per-packet key mixing), and CCMP/AES (CTR mode with CBC-MAC).

5. Radio PHY Type and Signal Quality: Analysis of physical radio specifications (802.11b, 802.11g, 802.11n, 802.11ac, 802.11ax) and signal quality percentages to assess physical link stability and proximity.

6. Wi-Fi Protected Setup (WPS) Status: Verification of whether external registrar PIN or push-button pairing is enabled, identifying brute-force risks.

7. Captive Portal Redirection State: Active verification of whether the network intercepts web traffic and enforces splash-screen authentication over unencrypted HTTP.

3.3 Hardware Querying vs. Manual Audit Capabilities

To maximize practical utility across varied auditing scenarios, the project incorporates two complementary evaluation modalities: automated hardware discovery and manual profile evaluation.

Automated Hardware Discovery operates in real time, interacting directly with the host operating system's wireless network adapter. It queries active connection telemetry via Windows `netsh wlan` and dispatches live HTTP probes to detect active captive portals. This mode provides zero-configuration auditing for the network to which the host machine is currently associated, as well as one-click environmental scanning of visible neighboring access points.

Manual Profile Evaluation provides an interactive audit modal within both the web dashboard and the terminal CLI. This capability allows security analysts, network engineers, and students to manually input hypothetical, remote, or proposed network configurations (e.g., testing an AP with WPA2-Personal, TKIP encryption, and WPS enabled). The system executes full vulnerability pattern matching and risk scoring on the submitted parameters without requiring physical proximity or actual connection to the target network. This dual modality expands the project's utility from a simple endpoint monitor into an effective instructional and policy-modeling tool.

3.4 Scoring Model, Risk Classification and Mitigation Guidelines

The quantitative evaluation scope centers on a deterministic mathematical scoring model that maps complex multi-attribute security states onto a continuous 0–100 scale. Zero represents a completely hardened wireless connection (utilizing WPA3-SAE, AES encryption, disabled WPS, and high-entropy credentials), whereas 100 represents severe exposure (such as an unencrypted open network with active captive portal redirection and cleartext credential transmission).

The model divides calculated scores into four distinct risk classifications: CRITICAL (scores 80–100), HIGH (scores 60–79), MEDIUM (scores 40–59), and LOW (scores 0–39). Each risk tier reflects the immediate likelihood and impact of exploitation. Crucially, the scope extends beyond passive risk identification: the engine dynamically generates prioritized, context-specific mitigation recommendations tailored to each identified vulnerability, advising users on protocol upgrades, cipher deprecation, WPS deactivation, and VPN encapsulation.

3.5 Tools, Technologies and Target Execution Environments

The implementation scope is built around the Python 3 programming language (specifically Python 3.8 and higher) executing in standard CPython runtime environments on Microsoft Windows 10 and Windows 11 host operating systems. The tool leverages native Windows administrative services, specifically the `netsh wlan` command-line helper, which provides deep interface instrumentation without requiring external kernel drivers or root-level raw socket privileges.

For application delivery, the system implements a Flask web server utilizing Jinja2 templating and custom CSS stylesheets to deliver a modern, dark-themed cybersecurity interface. Data persistence is achieved through structured JSON flat-file storage (`wifi_profile.json` and `users.json`), eliminating external relational database dependencies and ensuring portable, zero-configuration deployment.

The functional boundaries, capabilities, and explicit architectural exclusions of the Wi-Fi Security Profiler are summarized in Table 3.1.

Table 3.1: Functional Areas Covered and Architectural Boundaries

Functional Domain	In-Scope Capabilities	Out-of-Scope / Architectural Boundary
Interface Telemetry	Automated extraction of connected SSID, BSSID, Radio type, Cipher, Signal quality percentage	Low-level radio frequency spectrum modulation and PHY layer RF signal decoding
Captive Portal Check	Dual HTTP endpoint probing (captive.apple.com, msftconnecttest) and SSID keyword heuristics	Automated bypass of captive portal authentication or credential brute-forcing
Vulnerability Analysis	Deterministic rule engine detecting 6 critical risk patterns (Open, WEP, TKIP, WPS, Weak PW, Portal)	Exploit weaponization, automated packet injection, and dictionary wordlist attacks
Risk Assessment	Normalized 0-100 scoring model, mathematical penalty aggregation, 4 risk tiers (Critical, High, Med, Low)	Subjective qualitative estimation or unverified risk assertions
User Authentication	Multi-user signup, login, session guard, case-insensitive collision check, SHA-256 hashed storage	OAuth2/SAML third-party single sign-on or Active Directory federation

Functional Domain	In-Scope Capabilities	Out-of-Scope / Architectural Boundary
Interface Presentation	Full-featured Flask web UI with dark theme, manual audit modal, CSV/JSON export, and standalone CLI	Native compiled mobile applications (iOS/Android)
Execution Platform	Microsoft Windows 10 and Windows 11 host operating systems with native netsh wlan service	Direct execution on macOS and Linux without compatibility wrappers

3.6 Scope Limitations and Educational Application

To ensure strict compliance with legal, ethical, and academic boundaries, the Wi-Fi Security Profiler is intentionally engineered as a passive, defensive assessment utility. The system operates entirely within standard user-space permissions and does not engage in offensive exploitation or active network disruption.

Explicit functional limitations include:

1. Non-Promiscuous Operation: The tool does not place the wireless adapter into monitor or promiscuous mode, nor does it capture raw 802.11 radio frames or 802.1X management traffic, which would require third-party capture drivers like Npcap/WinPcap.

2. No Offensive Packet Injection: The software does not transmit 802.11 deauthentication frames, disassociation packets, or beacon floods, and does not perform active denial-of-service testing.

3. No Cryptographic Key Cracking: The system does not incorporate dictionary attack engines, rainbow table lookups, or GPU-accelerated hash cracking modules (e.g., Aircrack-ng or Hashcat) to recover pre-shared keys.

4. Host Operating System Specificity: Real-time hardware telemetry extraction is currently tailored to Microsoft Windows via `netsh wlan`. While manual auditing and the web interface are completely platform-independent, automated adapter querying requires Windows.

Within these deliberate boundaries, the Wi-Fi Security Profiler serves as an exemplary educational and practical tool for computer science students, network administrators, and cybersecurity professionals seeking to evaluate wireless security hygiene safely and rigorously.

CHAPTER 4 - SOFTWARE SPECIFICATIONS

4.1 Introduction

Software specifications establish the technical foundation, execution prerequisites, architectural dependencies, and programming paradigms required to build, execute, and maintain a software system. The Wi-Fi Security Profiler is engineered to combine low-overhead systems-level hardware inspection with high-level web visualization and quantitative security modeling.

This chapter outlines the runtime environment, the Python standard and third-party libraries employed, the core software engineering and Python programming concepts applied, the minimum hardware and operating system requirements, and the specific role of each architectural component in ensuring the dependable execution of the profiling system.

4.2 Programming Language and Development Environment

The system is implemented entirely in Python (version 3.8 or higher, tested and optimized across Python 3.10 through 3.12). Python was selected due to its expressive high-level syntax, strong cross-platform runtime support, extensive standard library for systems interaction and networking, and robust ecosystem for web application engineering.

The development lifecycle was conducted using Visual Studio Code and PyCharm Integrated Development Environments (IDEs), configured with Python virtual environments (`venv`) to ensure strict dependency isolation. Version control and task continuity were maintained using Git, with project architecture and module dependency relationships formally recorded in machine-readable format within `architecture.json`.

4.3 Python Libraries Used in the Project

The architectural design of the Wi-Fi Security Profiler deliberately prioritizes Python's standard library modules, minimizing external third-party dependencies to maximize software reliability, portability, and installation simplicity. The primary libraries and their roles are detailed below:

Flask (Version 3.1.3): The primary third-party dependency utilized in the project. Flask is a lightweight WSGI web application framework that manages HTTP request dispatching, session state cookies, flash notifications, Jinja2 template rendering, and RESTful API endpoints (`/api/detect`). Flask provides the backbone for the cybersecurity web dashboard without introducing heavy database ORM overhead.

Subprocess: Python standard library module utilized in `wifi_detector.py` to spawn operating system processes, specifically invoking Windows `netsh wlan show interfaces` and `netsh wlan show networks mode=bssid`. The `subprocess.run()` function is executed with explicit standard output capturing, standard error redirection, and text decoding, providing a secure, non-shell inter-process communication bridge.

Re (Regular Expressions): Standard library module facilitating compiled regex pattern matching. It parses the multiline, key-value formatted text output generated by `netsh`, reliably extracting SSIDs, BSSIDs, cipher suites, authentication standards, radio types, and signal strength percentages across varied language locales.

Hashlib: Standard library cryptographic hashing module used in `auth.py`. It implements deterministic SHA-256 (Secure Hash Algorithm 256-bit) one-way hashing to secure user passwords prior to persistence, preventing plaintext credential exposure.

Json: Standard library serialization module managing data persistence for `wifi_profile.json` (evaluated Wi-Fi audit profiles) and `users.json` (registered user credentials), utilizing atomic read and write operations.

Csv: Standard library module utilized in `app.py` to stream dynamically generated, comma-separated audit logs directly to user browsers via HTTP attachment headers.

Urllib.request: Standard library HTTP client utilized in `wifi_detector.py` to execute non-blocking HTTP probe requests against standardized captive portal detection endpoints, analyzing HTTP status codes and redirection URLs.

Unittest: Built-in automated testing framework used to implement a 29-test comprehensive test suite across `tests/`, ensuring complete functional verification of authentication, telemetry parsing, route handlers, and risk evaluation.

Table 4.1 outlines the primary libraries, their functional classification, and the specific programming subroutines utilized across the codebase.

Table 4.1: Python Libraries, Modules and Functional Purpose

Module Name	Primary Purpose	Specific Functions and Subroutines Utilized
flask	Web Application Delivery	Flask, render_template, request, redirect, url_for, session, flash, jsonify, Response
subprocess	System Command Execution	subprocess.run() with capture_output=True, text=True, check=False, shell=False
re	Textual Telemetry Extraction	re.search(), re.findall() with compiled multiline flags for netsh key-value matching
hashlib	Cryptographic Operations	hashlib.sha256() for hashing passwords into 64-character hexadecimal digests
json	Data Persistence	json.load(), json.dump() with indent formatting for wifi_profile.json and users.json
csv	Tabular Report Generation	csv.writer(), io.StringIO for streaming dynamic CSV audit logs to browser clients
urllib.request	Network Redirection Probing	urllib.request.urlopen() with timeout constraints to check captive portal HTTP responses
unittest	Automated Verification	unittest.TestCase, assertEqual, assertTrue, assertFalse, patch, mock

4.4 Python Programming Concepts

The project demonstrates the practical application of several advanced Python programming paradigms, software design patterns, and systems engineering concepts:

1. Modular Architecture and Separation of Concerns: The system is structured into five decoupled modules (`wifi_detector.py`, `risk_evaluator.py`, `auth.py`, `app.py`, and `main.py`). Each module encapsulates a single domain responsibility, communicating via standardized dictionary schemas and function return signatures, enabling independent testing, maintenance, and enhancement.

2. Inter-Process Communication (IPC): The application leverages `subprocess.run()` with `capture_output=True`, `text=True`, and `check=False`. By avoiding `shell=True`, the application prevents command injection vulnerabilities while safely capturing host networking telemetry.

3. Advanced Regular Expression Pattern Matching: Telemetry parsing utilizes compiled regular expressions with multiline flags (`re.MULTILINE`) to search for variable key-value pairs (e.g., `^\s*SSID\s*:\s*(.*)$`), handling whitespace variations and non-standard characters gracefully.

4. Cryptographic Hashing and Security Practices: Passwords are never stored in plaintext. The `auth.py` module applies `hashlib.sha256()` with UTF-8 encoding, producing 64-character hexadecimal digests. The registration logic enforces case-insensitive duplicate checks (`lower()`) and minimum length constraints.

5. Defensive Exception Handling and Resilience: All I/O operations, network HTTP requests, and process executions are wrapped in comprehensive `try...except` blocks with defensive fallbacks, ensuring that transient network errors, missing files, or disconnected adapters never crash the application.

6. Web Decorators and Session Management: The Flask web application employs `@app.route` decorators to map HTTP verbs to controller functions. User sessions are cryptographically signed using a persistent 32-byte secret key stored in `.secret_key`, and route handlers enforce authentication checks to protect sensitive audit and export routes.

The core Python programming concepts and their specific architectural applications are summarized in Table 4.2.

Table 4.2: Core Python Programming Concepts and Their System Application

Python Programming Concept	Implementation Mechanism	Application within Wi-Fi Security Profiler Architecture
Modular Architecture	Separate .py source files	Isolating detection, risk evaluation, auth, presentation, and CLI into discrete modules
Defensive Exception Handling	try / except / finally blocks	Preventing application crashes when Wi-Fi interface is disconnected or netsh fails

Python Programming Concept	Implementation Mechanism	Application within Wi-Fi Security Profiler Architecture
Inter-Process Communication	subprocess execution	Querying host OS networking subsystem without requiring kernel-level C extensions
String Parsing & Regular Expressions	Regular expression pattern buffers	Extracting telemetry metrics from structured localized Windows terminal output
Data Serialization & Dictionaries	Native dict objects and JSON files	Representing multi-attribute wireless profiles in uniform, extensible schema structures
Web Decorators & Session Management	@app.route decorators and session dict	Managing HTTP request lifecycles, user authentication state, and endpoint guards
Unit Testing & Mocking	unittest and mock libraries	Simulating diverse network configurations to verify risk scoring and route behavior

4.5 Software Requirements and Testing Environment

To ensure reliable performance, the system was developed and validated against well-defined software and hardware specifications. The application runs smoothly on standard modern personal computers and laptops without requiring dedicated graphics processors or specialized network hardware.

Hardware Requirements: The minimum host system configuration requires an x86-64 dual-core processor (Intel Core i3 or AMD Ryzen 3), 2.0 GB of RAM (4.0 GB recommended), 100 MB of free storage, and an integrated or USB wireless network adapter compliant with IEEE 802.11b/g/n/ac/ax.

Software Requirements: The software requires Microsoft Windows 10 (64-bit) or Windows 11 (64-bit) with the native WLAN AutoConfig service active, Python 3.8 or higher installed and added to the system PATH, and any modern web browser (Google Chrome, Microsoft Edge, Mozilla Firefox) supporting HTML5, CSS3, and JavaScript.

The complete software requirements and testing environment parameters are detailed in Table 4.3.

Table 4.3: Minimum Software, Hardware and Runtime Specifications

System Component	Minimum Specification	Recommended Specification	Notes / Function in Execution
Host Processor (CPU)	1.6 GHz dual-core x86-64	2.4 GHz quad-core or higher	Executes Python runtime and regex parsing
System Memory (RAM)	2.0 GB RAM	4.0 GB RAM or higher	Buffers Flask server and network telemetry
Wireless Adapter (NIC)	IEEE 802.11b/g/n adapter	IEEE 802.11ac or 802.11ax adapter	Required for hardware netsh telemetry queries
Operating System	Microsoft Windows 10 (64-bit)	Microsoft Windows 11 (64-bit)	Native host platform utilizing netsh wlan
Python Runtime	Python 3.8.x	Python 3.10.x - 3.12.x	CPython interpreter executing application logic
Web Browser	Chromium 90+, Firefox 88+	Latest Chrome, Firefox, or Edge	Renders dark-mode glassmorphism dashboard UI
Disk Storage	100 MB free space	500 MB free space	Stores application source, JSON databases, logs

4.6 Role of Software Components in Project Execution

Each software component plays a distinct and coordinated role throughout system execution. When a user initiates hardware detection, `app.py` invokes `detect_connected_wifi()` in `wifi_detector.py`. This component executes `netsh wlan show interfaces` via `subprocess`, parses the resulting output using `re`, dispatches an HTTP probe via `urllib.request` to test for captive portal redirects, and returns a structured dictionary.

The dictionary is immediately passed to `risk_evaluator.py`, where `evaluate_wifi_risk()` identifies matching vulnerability patterns, computes the normalized risk score, assigns the risk tier, and generates remediation recommendations. The evaluated profile is serialized to `wifi_profile.json` using `json.dump()`. Finally, `app.py` passes the profile data into Jinja2 templates (`dashboard.html`), where CSS styles render the appropriate neon badges and metric cards, completing the seamless execution pipeline.

CHAPTER 5 – WORKFLOW OF THE PROJECT

5.1 Introduction

The workflow of the Wi-Fi Security Profiler defines the chronological and algorithmic pipeline through which raw physical wireless signals and operating system telemetry are captured, structured, evaluated against vulnerability rules, quantified into risk metrics, and presented to end users. A transparent, deterministic workflow ensures that every evaluated network undergoes rigorous, repeatable security auditing.

This chapter details the overarching system architecture and walks through each sequential phase of execution: Phase 1 (Telemetry Acquisition & Environmental Detection), Phase 2 (Vulnerability Analysis & Quantitative Risk Scoring), and Phase 3 (Web Visualization, User Session Management & Audit Reporting).

5.2 Overall Project Workflow

The system workflow operates as a pipeline of decoupled processing stages. Execution begins with environment sensing, transitions through rule-based risk evaluation, persists structured state to local storage, and culminates in interactive presentation and export capabilities.

The workflow is identical whether triggered from the Flask web interface or the terminal CLI, ensuring complete functional parity across operational modes. Table 5.1 provides a structured overview of the execution stages, primary activities, data transformations, and resulting system outputs.

Table 5.1: Multi-Phase Processing Workflow of Wi-Fi Security Profiler

Execution Phase	Primary System Activities	Data Transformations & Operations	Generated Output / System State
Phase 1: Acquisition	Execute netsh wlan show interfaces; Probe captive portal endpoints	Capture stdout, decode UTF-8, extract SSID, BSSID, Auth, Cipher, Signal; HTTP probe	Normalized raw Wi-Fi profile dictionary structure
Phase 2: Evaluation	Feed profile dictionary into risk_evaluator engine	Match 6 vulnerability patterns; Calculate additive penalties; Compute 0-100 score; Tier risk	Evaluated profile with risk_score, risk_level, vulnerabilities, recommendations

Execution Phase	Primary System Activities	Data Transformations & Operations	Generated Output / System State
Phase 3: Persistence	Check for existing profile records in wifi_profile.json	Update existing entry or append new profile record; Serialize with atomic file write	Updated persistent JSON repository on local storage
Phase 4: Presentation	Dispatch profile data to Flask web dashboard or CLI terminal	Render neon risk badges, summary cards, and tabular metrics; Handle user audit modal	Interactive web dashboard view or formatted terminal display
Phase 5: Reporting	User requests export via /export/<format_type> endpoint	Iterate profile records, construct sanitized CSV table or JSON payload, attach headers	Downloadable audit report file (.csv or .json format)

5.3 Phase 1 – Telemetry Acquisition and Environmental Detection

Phase 1 is responsible for environmental discovery and hardware telemetry extraction. The process commences when the user clicks 'Auto Detect & Add' on the web dashboard or selects the detection option in the terminal CLI.

The `wifi_detector` module executes `subprocess.run(['netsh', 'wlan', 'show', 'interfaces'], capture_output=True, text=True)`. The operating system queries the active wireless adapter via the WLAN AutoConfig API and returns a multiline textual summary. The module decodes the standard output stream using UTF-8 (falling back to CP437 if localized Windows encoding is encountered) and applies compiled regular expressions to extract key attributes: SSID, BSSID, State, Radio type, Authentication, Cipher, and Signal percentage.

If the adapter is disconnected or wireless radio is turned off, the module detects the absence of an active connection and returns an empty profile safely without raising unhandled exceptions.

Simultaneously, the module initiates captive portal verification via `check_captive_portal()`. It constructs an HTTP GET request to `http://captive.apple.com/hotspot-detect.html` with a short 3.0-second timeout. If the gateway intercepts the request and issues an HTTP 302/301 redirect or returns a response body that does not contain the expected `<HTML><HEAD><TITLE>Success</TITLE></HEAD><BODY>Success</BODY></HTML>` token, the network is flagged with `captive_portal = 'Detected'`. As a defensive fallback, the module

also applies lexical matching against the SSID for keywords like 'Guest', 'Airport', 'Hotel', or 'Free', ensuring reliable detection across varied network topologies.

5.4 Phase 2 – Vulnerability Analysis and Quantitative Risk Scoring

In Phase 2, the raw telemetry dictionary generated in Phase 1 is delivered to the `risk_evaluator` engine. The engine executes a multi-step analytical procedure to identify vulnerabilities, calculate mathematical penalty deductions, compute a normalized risk score, and determine the appropriate risk tier.

Step 1: Pattern Matching (`identify_vulnerability_patterns`). The engine interrogates the profile against six distinct vulnerability rules:

- `OPEN_NETWORK`: Triggered if `security == 'Open'` or `encryption == 'None'`. Flagged as CRITICAL severity, indicating cleartext transmission and vulnerability to wireless packet sniffing.

- `CAPTIVE_PORTAL`: Triggered if `captive_portal == 'Detected'`. Flagged as HIGH severity, identifying unencrypted web authentication and credential interception risks.

- `WEP_DEPRECATED`: Triggered if `security == 'WEP'`. Flagged as CRITICAL severity due to weak RC4 stream cipher and rapid statistical key recovery.

- `TKIP_LEGACY`: Triggered if `encryption == 'TKIP'`. Flagged as HIGH severity due to known packet injection vulnerabilities and protocol deprecation.

- `WPS_ENABLED`: Triggered if `wps == 'Enabled'`. Flagged as HIGH severity due to external registrar PIN brute-force susceptibility.

- `WEAK_PASSWORD`: Triggered if `password_strength` is 'Weak' or 'Very Weak'. Flagged as MEDIUM severity due to offline dictionary cracking exposure.

Step 2: Mathematical Risk Scoring (`evaluate_wifi_risk`). The engine establishes a baseline security score based on the underlying authentication and encryption protocol: WPA3 initializes at 10, WPA2-AES initializes at 25, WPA-TKIP initializes at 60, WEP initializes at 85, and Open networks initialize at 95.

Step 3: Penalty Adjustment and Clamping. For each identified vulnerability pattern, additive penalty points are applied (+25 for captive portal, +20 for active WPS, +15 for weak password, etc.). Furthermore, password strength modifiers (-10 for Strong, +10 for Weak) adjust the score. The final composite score is clamped strictly between 0 and 100:

Score = min(100, max(0, Base_Score + Sum(Penalties) + Modifier))

Step 4: Classification and Recommendations. The score maps to a risk tier: CRITICAL (80–100), HIGH (60–79), MEDIUM (40–59), or LOW (0–39). Finally, `generate_recommendations()` creates customized defensive countermeasures tailored to each detected vulnerability.

5.5 Phase 3 – Web Visualization, User Session Management and Audit Reporting

In Phase 3, the evaluated profile is returned to `app.py`. The controller loads `wifi_profile.json`, checks if a profile with matching SSID and BSSID already exists, updates or appends the record, and writes the updated list back to disk.

The user's browser is redirected to `/dashboard`. Flask renders `dashboard.html` with Jinja2, populating the connection monitor with live telemetry (SSID, BSSID, Signal bar, Protocol badge, Risk badge), updating summary statistics (Total Networks, Critical Alerts, Average Risk Score), and displaying persistent profile cards.

If the user wishes to test a hypothetical network or evaluate a remote router, they open the 'Manual Audit Modal', input custom parameters (SSID, Security, Encryption, WPS, Password Strength, Captive Portal), and submit the form. The `/manual_add` route evaluates the configuration and immediately adds it to the dashboard.

Finally, when compliance records or offline documentation are required, the user clicks 'Export JSON' or 'Export CSV'. The `/export/<format_type>` route serializes the audit database, attaches `Content-Disposition: attachment; filename=wifi_security_audit_report.<ext>` headers, and streams the file directly to the client's download manager.

CHAPTER 6 – MODULES OF THE PROJECT

6.1 Introduction

A modular software architecture decomposes complex systems into independent, reusable, and loosely coupled components. Each module encapsulates a cohesive set of functions, data models, and business logic, interacting with other modules exclusively through well-defined interfaces.

The Wi-Fi Security Profiler is organized into five core modules: `wifi_detector.py` (Hardware & Telemetry Detection), `risk_evaluator.py` (Vulnerability & Risk Analysis Engine), `auth.py` (User Management & Cryptographic Authentication), `app.py` (Flask Web Controller & Route Handlers), and `main.py` (Terminal CLI Application). This chapter analyzes the internal implementation, functional responsibilities, and integration of each module, concluding with four evaluated real-world case studies.

6.2 Module 1 – Wi-Fi Detector Module (wifi_detector.py)

The `wifi_detector.py` module serves as the hardware abstraction and environmental interrogation layer. It is responsible for communicating with the host operating system's networking stack, executing wireless queries, parsing terminal output, and probing network connectivity.

The module provides five primary functions:

1. `detect_connected_wifi()`: Executes `netsh wlan show interfaces` via `subprocess.run()`. It extracts the active connection's SSID, BSSID, radio type, state, signal quality, authentication, and cipher suite using regular expressions. It invokes captive portal verification and returns a normalized dictionary.

2. `check_captive_portal(ssid)`: Implements dual-stage captive portal detection, executing an HTTP probe to `http://captive.apple.com/hotspot-detect.html` with a 3.0-second timeout, falling back to heuristic keyword inspection of the SSID string.

3. `parse_netsh_auth_and_cipher(auth_str, cipher_str)`: Normalizes diverse Windows netsh localization strings (such as 'WPA2-Personal', 'WPA2-Enterprise', 'Open', 'CCMP', 'TKIP', 'None') into uniform standardized protocol identifiers.

4. `scan_nearby_networks()`: Invokes `netsh wlan show networks mode=bssid`, parsing nearby access points to extract visible SSIDs, signal percentages, authentication types, and ciphers for comparative environmental analysis.

5. `get_profile_password_strength(ssid)`: Queries stored Windows WLAN profiles using `netsh wlan show profile name=... key=clear` to evaluate key length and character complexity if user administrative privileges permit.

6.3 Module 2 – Risk Evaluator Engine (risk_evaluator.py)

The `risk_evaluator.py` module encapsulates the core analytical and mathematical logic of the application. It operates purely on data dictionaries, containing zero operating system or web framework dependencies, which ensures maximum execution speed and flawless testability.

The module contains five key functions:

1. `identify_vulnerability_patterns(wifi_profile)`: Evaluates profile attributes against the six vulnerability rules (`OPEN_NETWORK`, `CAPTIVE_PORTAL`, `WEP_DEPRECATED`, `TKIP_LEGACY`, `WPS_ENABLED`, `WEAK_PASSWORD`), returning a list of vulnerability objects containing unique IDs, severity ratings, titles, and technical descriptions.

2. `classify_risk_level(score)`: Maps an integer score between 0 and 100 to one of four discrete risk tiers: CRITICAL (>= 80), HIGH (>= 60), MEDIUM (>= 40), or LOW (< 40).

3. `generate_recommendations(wifi_profile, vulnerabilities)`: Dynamically generates actionable security hardening steps tailored to the detected vulnerabilities, advising users on VPN usage, cipher deprecation, and AP re-configuration.

4. `evaluate_wifi_risk(wifi_profile)`: The master evaluation pipeline function. It invokes pattern matching, calculates the composite risk score, assigns the risk tier, generates recommendations, and enriches the profile dictionary with complete audit results.

5. `batch_process_profiles(profiles_list)`: Iterates through a collection of wireless profiles, computing aggregate metrics including total networks analyzed, distribution across risk tiers, average risk score, and counts of critical alerts.

6.4 Module 3 – Authentication & Security Module (auth.py)

The `auth.py` module manages user authentication, account registration, credential persistence, and cryptographic verification for the web application.

The module maintains an independent JSON data store (`users.json`). To protect credentials against unauthorized exposure, passwords are never stored in cleartext. The module utilizes Python's `hashlib.sha256()` function, encoding user passwords to UTF-8 and generating a deterministic 64-character hexadecimal digest.

The module implements four core functions:

1. `register_user(username, password)`: Enforces input sanitization, whitespace trimming, and minimum length constraints (minimum 3 characters for username, minimum 4 characters for password). It performs case-insensitive duplicate checks to prevent username spoofing and stores the username and hashed password atomically in `users.json`.

2. `validate_user(username, password)`: Hashes the incoming password candidate using SHA-256 and compares the resulting digest against the stored hash for the matching username, returning True upon successful authentication and False otherwise.

3. `load_users()` and `save_users(users)`: Helper routines that manage safe serialization and deserialization of the user database with defensive exception handling.

6.5 Module 4 – Web Application & Presentation Controller (app.py)

The `app.py` module serves as the primary controller and HTTP presentation layer. It initializes the Flask WSGI server, configures session cookie encryption using a persistent 32-byte secret key stored in `.secret_key`, and defines application routes.

The module implements route handlers for user authentication (`/login`, `/signup`, `/logout`), dashboard presentation (`/dashboard`), RESTful hardware telemetry (`/api/detect`), automated detection ingestion (`/auto_detect_and_add`), nearby network scanning (`/scan_and_add`), manual audit modal submissions (`/manual_add`), profile deletion (`/delete_profile/<path:ssid>`), and audit report exports (`/export/<format_type>`).

The frontend presentation is powered by Jinja2 templates (`home.html`, `login.html`, `dashboard.html`) and a specialized stylesheet (`static/css/style.css`). The styling establishes a modern cybersecurity dark theme with high-contrast typography, glassmorphism cards, glowing risk badges, and accessible modal dialogs.

6.6 Module 5 – Terminal Command Line Interface (main.py)

The `main.py` module provides a standalone, interactive Command Line Interface (CLI) that allows users and security auditors to utilize all profiling capabilities in headless, console, or remote SSH environments without running a web server.

The CLI displays an ASCII art banner, provides an interactive numbered menu, formats live telemetry and nearby access point scans into clean terminal tables, allows manual profile evaluation with real-time feedback, and presents aggregate risk summaries. Both `app.py` and `main.py` rely on the identical

`risk_evaluator` and `wifi_detector` modules, ensuring complete consistency of audit findings across interfaces.

To demonstrate the analytical capabilities of the system, four representative case studies spanning diverse wireless security configurations were evaluated through the profiling pipeline. The resulting telemetry, detected vulnerability patterns, risk scores, and generated recommendations are presented in Table 6.1.

Table 6.1: Evaluated Wi-Fi Security Audit Case Studies

Network Profile / Scenario	Observed Security Attributes	Detected Vulnerabilities	Assigned Score & Risk Tier	Primary Hardening Recommendation
Public Campus Guest Wi-Fi	Auth: Open, Cipher: None, Captive Portal: Detected, WPS: Disabled	OPEN_NETWORK, CAPTIVE_PORTAL	Score: 75 Risk: HIGH	Enforce end-to-end VPN tunnel; Avoid unencrypted credentials
Legacy Laboratory Network	Auth: WEP, Cipher: WEP40, Captive Portal: None, WPS: Disabled	WEP_DEPRECATED	Score: 90 Risk: CRITICAL	Immediately retire WEP; Upgrade hardware to WPA2-AES or WPA3
Residential Home Wi-Fi	Auth: WPA2-Personal, Cipher: TKIP, WPS: Enabled, Password: Weak	TKIP_LEGACY, WPS_ENABLED, WEAK_PASSWORD	Score: 60 Risk: HIGH	Disable WPS in AP admin; Switch cipher to AES; Set strong passphrase
Hardened Enterprise Wi-Fi	Auth: WPA3-Personal, Cipher: AES, Captive Portal: None, WPS: Disabled	None (Zero vulnerabilities)	Score: 10 Risk: LOW	Maintain firmware currency; Regularly audit authorized client devices

6.7 Integration and Overall Outcome of the Modules

The integration between the five modules achieves a clean, fault-tolerant architecture. Data flows unidirectionally from the environmental sensing module (`wifi_detector.py`) to the analytical engine

(`risk_evaluator.py`), and subsequently to the presentation and persistence controllers (`app.py` and `main.py`). Authentication is enforced at the controller perimeter by `auth.py`, ensuring that sensitive audit records and export capabilities remain accessible only to authorized users.

Because each module maintains strict boundaries and independent responsibilities, changes to the operating system command syntax or web presentation framework do not impact the core risk scoring mathematics. This modularity ensures outstanding code maintainability, simplifies automated unit testing, and facilitates seamless future enhancements.

CHAPTER 7 – SOURCE CODE

7.1 Introduction

The source code architecture of the Wi-Fi Security Profiler reflects the practical translation of cybersecurity engineering principles into maintainable, efficient Python implementations. Each module is structured to enforce separation of concerns, defensive programming, comprehensive exception handling, and adherence to clean code conventions.

This chapter presents and analyzes the core implementation algorithms and representative source code structures across the five major functional components: hardware telemetry extraction (`wifi_detector.py`), vulnerability pattern matching and mathematical scoring (`risk_evaluator.py`), cryptographic user authentication (`auth.py`), web route controllers and streaming report exports (`app.py`), and the automated testing suite (`tests/`).

7.2 Core Telemetry and Captive Portal Detection Implementation

The hardware telemetry module (`wifi_detector.py`) provides the system's interface to the host operating system's wireless subsystem. It executes administrative queries via `subprocess`, parses complex multiline output with regular expressions, and performs active HTTP network probing.

The core telemetry acquisition logic is encapsulated in `detect_connected_wifi()` and `check_captive_portal()`:

Code Listing 7.1: Telemetry Extraction and Captive Portal Probing (wifi_detector.py)

```
import subprocess
import re
import urllib.request

def check_captive_portal(ssid=""):
    """Probes standard HTTP endpoints to detect captive portal redirection."""
    portal_urls = [
        "http://captive.apple.com/hotspot-detect.html",
        "http://www.msftconnecttest.com/connecttest.txt"
    ]
    for url in portal_urls:
        try:
            req = urllib.request.Request(url, headers={"User-Agent":
"Mozilla/5.0"})
            with urllib.request.urlopen(req, timeout=3.0) as resp:
```

```
                content = resp.read().decode("utf-8", errors="ignore").strip()
                if "Success" in content or "Microsoft Connect Test" in content:
                    return "None"
                return "Detected"
        except Exception:
            pass
    # Heuristic fallback matching common public/guest SSID keywords
    public_keywords = ["guest", "free", "airport", "hotel", "cafe", "campus"]
    if any(kw in ssid.lower() for kw in public_keywords):
        return "Detected"
    return "None"

def detect_connected_wifi():
    """Queries Windows netsh wlan interfaces to extract live wireless
telemetry."""
    try:
        res = subprocess.run(["netsh", "wlan", "show", "interfaces"],
                             capture_output=True, text=True, check=False)
        out = res.stdout
        if not out or "State" not in out:
            return {}
        state_m = re.search(r"^\s*State\s*:\s*(.+)$", out, re.M)
        if not state_m or "connected" not in state_m.group(1).lower():
            return {}
        ssid_m = re.search(r"^\s*SSID\s*:\s*(.+)$", out, re.M)
        bssid_m = re.search(r"^\s*BSSID\s*:\s*(.+)$", out, re.M)
        auth_m = re.search(r"^\s*Authentication\s*:\s*(.+)$", out, re.M)
        cipher_m = re.search(r"^\s*Cipher\s*:\s*(.+)$", out, re.M)
        signal_m = re.search(r"^\s*Signal\s*:\s*(\d+)%", out, re.M)

        ssid = ssid_m.group(1).strip() if ssid_m else "Unknown"
        portal = check_captive_portal(ssid)
        return {
            "ssid": ssid,
            "bssid": bssid_m.group(1).strip() if bssid_m else "00:00:00:00:00:00",
            "security": auth_m.group(1).strip() if auth_m else "Open",
            "encryption": cipher_m.group(1).strip() if cipher_m else "None",
            "signal_quality": signal_m.group(1).strip() if signal_m else "75",
            "captive_portal": portal,
            "wps": "Disabled"
```

```
        }
    except Exception:
        return {}
```

Technical Analysis of Code Listing 7.1: The `check_captive_portal()` function dispatches an HTTP GET request to `http://captive.apple.com/hotspot-detect.html`. When an open wireless access point enforces an internal web captive portal, the access gateway intercepts the outbound port 80 request and returns an HTTP 302 redirect or an HTML splash page. The code inspects the decoded response body; if the expected 'Success' string is absent, the network is flagged as 'Detected'. The lexical heuristic fallback evaluates the SSID against known public Wi-Fi keywords, ensuring robust identification even when DNS resolution is intercepted. The `detect_connected_wifi()` function executes `netsh wlan show interfaces` safely without `shell=True`, preventing command injection, and uses compiled multiline regex patterns to extract interface parameters into a standardized dictionary.

7.3 Vulnerability Pattern Matching and Risk Scoring Engine

The `risk_evaluator.py` module contains the rule-based analytical engine. It evaluates input profile dictionaries against predefined security rules, applies mathematical scoring penalties, maps the resulting score to a risk tier, and generates actionable hardening recommendations.

The pattern detection and scoring algorithms are implemented as follows:

Code Listing 7.2: Vulnerability Detection and Risk Scoring (risk_evaluator.py)

```
def identify_vulnerability_patterns(wifi_profile):
    """Evaluates profile attributes against 6 distinct vulnerability
signatures."""
    vulnerabilities = []
    sec = wifi_profile.get("security", "").strip()
    enc = wifi_profile.get("encryption", "").strip()
    wps_stat = wifi_profile.get("wps", "").strip()
    pw_str = wifi_profile.get("password_strength", "").strip()
    portal_val = wifi_profile.get("captive_portal", "").strip()

    if sec == "Open" or enc == "None":
        vulnerabilities.append({
            "id": "OPEN_NETWORK", "severity": "CRITICAL",
            "title": "Unencrypted Open Network",
            "description": "Traffic is unencrypted and vulnerable to eavesdropping
and MITM sniffing."
```

```
        })
    if portal_val == "Detected":
        vulnerabilities.append({
            "id": "CAPTIVE_PORTAL", "severity": "HIGH",
            "title": "Internal Web Login / Captive Portal Detected",
            "description": "Unencrypted splash login exposes credentials to over-
the-air interception."
        })
    if sec == "WEP":
        vulnerabilities.append({
            "id": "WEP_DEPRECATED", "severity": "CRITICAL",
            "title": "Obsolete WEP Encryption",
            "description": "Uses weak RC4 cipher that can be cracked within
minutes via IV collisions."
        })
    if enc == "TKIP":
        vulnerabilities.append({
            "id": "TKIP_LEGACY", "severity": "HIGH",
            "title": "Legacy TKIP Cipher",
            "description": "Deprecated cipher vulnerable to keystream recovery and
packet injection."
        })
    if wps_stat == "Enabled":
        vulnerabilities.append({
            "id": "WPS_ENABLED", "severity": "HIGH",
            "title": "WPS PIN Vulnerability Active",
            "description": "PIN authentication allows offline brute-forcing via
Reaver/Pixie Dust."
        })
    if pw_str in ["Weak", "Very Weak"]:
        vulnerabilities.append({
            "id": "WEAK_PASSWORD", "severity": "MEDIUM",
            "title": "Insufficient Passphrase Entropy",
            "description": "Pre-shared key is susceptible to dictionary
computation once handshake is captured."
        })
    return vulnerabilities


def evaluate_wifi_risk(wifi_profile):
    """Computes composite 0-100 risk score and assigns risk classification
tier."""
    vulns = identify_vulnerability_patterns(wifi_profile)
```

```
    sec = wifi_profile.get("security", "").strip()
    enc = wifi_profile.get("encryption", "").strip()
    pw_str = wifi_profile.get("password_strength", "").strip()

    # Baseline score based on cryptographic protocol
    base_score = 10
    if "WPA3" in sec:
        base_score = 10
    elif "WPA2" in sec and enc in ["AES", "CCMP"]:
        base_score = 25
    elif "WPA" in sec or enc == "TKIP":
        base_score = 60
    elif sec == "WEP":
        base_score = 85
    elif sec == "Open" or enc == "None":
        base_score = 95

    # Penalty adjustments for active vulnerabilities
    penalty = sum(25 if v["id"] in ["OPEN_NETWORK", "CAPTIVE_PORTAL",
"WEP_DEPRECATED"] else 20 for v in vulns)
    pw_mod = -10 if pw_str == "Strong" else (10 if pw_str in ["Weak", "Very Weak"]
else 0)
    final_score = min(100, max(0, int(base_score * 0.5 + penalty * 0.5 + pw_mod)))

    # Risk tier classification
    tier = "CRITICAL" if final_score >= 80 else ("HIGH" if final_score >= 60 else
("MEDIUM" if final_score >= 40 else "LOW"))
    result = dict(wifi_profile)
    result["risk_score"] = final_score
    result["risk_level"] = tier
    result["vulnerabilities"] = vulns
    result["recommendations"] = generate_recommendations(wifi_profile, vulns)
    return result
```

Technical Analysis of Code Listing 7.2: The `identify_vulnerability_patterns()` function inspects the profile dictionary and populates an array of vulnerability objects containing severity ratings and technical impact statements. The `evaluate_wifi_risk()` function combines the baseline security score with the weighted penalty points, applies the password strength modifier, and clamps the result between 0 and 100. By balancing baseline protocol weights with specific vulnerability penalties, the formula

ensures that high-risk configurations (such as WEP or unencrypted portals) consistently register in the `CRITICAL` or `HIGH` tiers, while fully hardened WPA3 networks achieve `LOW` risk ratings.

7.4 Cryptographic User Authentication and Data Persistence

The authentication module (`auth.py`) implements secure account registration, credential persistence, and login validation for the multi-user web dashboard. It avoids external database dependencies by maintaining a clean JSON flat-file repository (`users.json`).

The cryptographic hashing and registration logic are implemented as follows:

Code Listing 7.3: SHA-256 Authentication and User Validation (auth.py)

```
import hashlib
import json
import os

USERS_FILE = "users.json"

def hash_password(password):
    """Generates a deterministic 64-character SHA-256 hexadecimal digest."""
    return hashlib.sha256(password.strip().encode("utf-8")).hexdigest()

def register_user(username, password):
    """Registers a new user enforcing length constraints and collision
prevention."""
    username = username.strip()
    password = password.strip()
    if len(username) < 3 or len(password) < 4:
        return False, "Username must be >= 3 chars and password >= 4 chars."
    users = load_users()
    # Prevent case-insensitive duplicate username collision attacks
    if any(u.lower() == username.lower() for u in users):
        return False, "Username already exists. Please choose a different
username."
    users[username] = hash_password(password)
    save_users(users)
    return True, "Registration successful. You may now sign in."

def validate_user(username, password):
    """Validates login credentials against stored SHA-256 hashes."""
    username = username.strip()
```

```
    password = password.strip()
    users = load_users()
    target_hash = users.get(username)
    if not target_hash:
        return False
    return target_hash == hash_password(password)
```

Technical Analysis of Code Listing 7.3: The `hash_password()` function converts the trimmed plaintext password string into UTF-8 bytes and computes its SHA-256 digest via `hashlib.sha256()`. The resulting 64-character hexadecimal representation ensures that plaintext credentials never reside on persistent storage. In `register_user()`, defensive checks enforce minimum length thresholds to eliminate trivial credentials, and a case-insensitive check (`u.lower() == username.lower()`) prevents collision attacks where an attacker registers 'Admin' alongside an existing 'admin'. Validating credentials in `validate_user()` re-hashes the input and performs direct hash comparison, returning a boolean authentication result.

7.5 Flask Application Controller and Export Handlers

The `app.py` module integrates all system services into an interactive web application. It configures Flask, handles session management, validates route access, processes manual audit modal submissions, and streams dynamic audit reports.

The manual audit ingestion and streaming CSV/JSON export handlers are implemented as follows:

Code Listing 7.4: Route Handlers and Report Export Engine (app.py)

```
from flask import Flask, render_template, request, redirect, url_for, session,
flash, Response, jsonify
import csv
import io
import json
from risk_evaluator import evaluate_wifi_risk

app = Flask(__name__)

@app.route("/manual_add", methods=["POST"])
def manual_add():
    """Handles manual audit modal submission for hypothetical configurations."""
    if "user" not in session:
        return redirect(url_for("login"))
```

```
    ssid = request.form.get("ssid", "").strip()
    if not ssid:
        flash("SSID name cannot be blank for manual audit.", "error")
        return redirect(url_for("dashboard"))
    raw_profile = {
        "ssid": ssid,
        "bssid": request.form.get("bssid", "00:11:22:33:44:55").strip(),
        "security": request.form.get("security", "WPA2-Personal"),
        "encryption": request.form.get("encryption", "AES"),
        "signal_quality": request.form.get("signal_quality", "80"),
        "captive_portal": request.form.get("captive_portal", "None"),
        "wps": request.form.get("wps", "Disabled"),
        "password_strength": request.form.get("password_strength", "Medium")
    }
    evaluated = evaluate_wifi_risk(raw_profile)
    save_profile_to_db(evaluated)
    flash(f"Audited profile '{ssid}' evaluated with Risk Level:
{evaluated['risk_level']}", "success")
    return redirect(url_for("dashboard"))


@app.route("/export/<format_type>")
def export_report(format_type):
    """Generates and streams downloadable audit logs in CSV or JSON format."""
    if "user" not in session:
        return redirect(url_for("login"))
    profiles = load_profiles_from_db()
    if format_type.lower() == "json":
        return Response(json.dumps(profiles, indent=2),
                        mimetype="application/json",
                        headers={"Content-Disposition":
"attachment;filename=wifi_audit_report.json"})
    elif format_type.lower() == "csv":
        si = io.StringIO()
        cw = csv.writer(si)
        cw.writerow(["SSID", "BSSID", "Security", "Encryption", "Signal",
"Portal", "WPS", "Score", "Risk Tier"])
        for p in profiles:
            cw.writerow([p.get("ssid"), p.get("bssid"), p.get("security"),
p.get("encryption"),
                         p.get("signal_quality"), p.get("captive_portal"),
p.get("wps"),
```

```
                        p.get("risk_score"), p.get("risk_level")])
        return Response(si.getvalue(),
                        mimetype="text/csv",
                        headers={"Content-Disposition":
"attachment;filename=wifi_audit_report.csv"})
```

Technical Analysis of Code Listing 7.4: The `/manual_add` controller enforces an authentication session guard before processing incoming form fields. It extracts the security parameters submitted through the modal dialog, passes the constructed dictionary to `evaluate_wifi_risk()`, commits the scored record to `wifi_profile.json`, and flashes a notification to the user interface. The `/export/<format_type>` route provides on-demand reporting. For JSON exports, it streams formatted text with `application/json` MIME headers; for CSV exports, it uses Python's `io.StringIO` and `csv.writer` to serialize audit metrics dynamically in-memory, attaching `Content-Disposition: attachment` headers to trigger a browser file download without creating temporary clutter on the server disk.

7.6 Automated Test Suite and Verification

To ensure software reliability, functional correctness, and regression resilience, the codebase includes an extensive automated test suite built on Python's native `unittest` framework. The suite contains 29 comprehensive test cases distributed across four test modules:

1. `tests/test_risk_evaluator.py`: 8 test cases validating vulnerability pattern identification, baseline protocol scoring, penalty calculations, boundary clamping, and aggregate batch metrics.

2. `tests/test_auth.py`: 6 test cases verifying SHA-256 hashing consistency, valid registrations, whitespace stripping, rejection of short credentials, case-insensitive collision prevention, and wrong password rejections.

3. `tests/test_wifi_detector.py`: 7 test cases validating netsh localization parsing, open/WEP/WPA2/WPA3 mappings, graceful handling of disconnected adapters, and captive portal keyword heuristics.

4. `tests/test_app.py`: 8 test cases verifying Flask endpoint status codes, unauthenticated session redirects, profile deletion, manual modal audits, and CSV/JSON export downloads.

Execution of the automated test runner confirms 100% test passage:

Command: python -m unittest discover -s tests -p 'test_*.py' -v

Results: Ran 29 tests in 8.841s -- OK (All test cases passed with zero failures or errors).

CHAPTER 8 – FEATURES OF THE PROJECT

8.1 Introduction

The Wi-Fi Security Profiler incorporates an extensive suite of defensive capabilities designed to simplify wireless security evaluation, automate telemetry extraction, and provide actionable security intelligence.

This chapter highlights the core technical features of the application, encompassing automated hardware detection, dual-stage captive portal discovery, transparent mathematical risk scoring, cryptographic user management, interactive web dashboard controls, and dual operational modalities.

8.2 Automated Hardware Telemetry and Live Network Detection

The application provides zero-configuration, real-time wireless telemetry extraction directly from the host computer's active network interface card. By issuing non-shell subcommands to Windows `netsh wlan`, the tool extracts the serving access point's SSID, BSSID (MAC address), radio physical standard (802.11ax/ac/n), signal quality percentage, authentication protocol, and encryption cipher.

Additionally, the tool includes a nearby access point scanner that enumerates all visible wireless networks within physical radio range, allowing users to benchmark the security hygiene of neighboring access points against their own.

8.3 Dual-Stage Captive Portal / Web Splash Page Detection

Public and institutional Wi-Fi networks frequently present deceptive security characteristics, associating clients without link-layer encryption while requiring credentials on an internal HTTP splash page. The Wi-Fi Security Profiler addresses this challenge through dual-stage detection.

Stage one executes active HTTP probe requests to standardized endpoints (`http://captive.apple.com/hotspot-detect.html`), analyzing response status codes and payload contents for redirection signatures. Stage two applies lexical heuristic analysis to identify public Wi-Fi keywords in the SSID, ensuring accurate detection even when client internet connectivity is restricted.

8.4 Transparent 0-100 Risk Scoring and Actionable Hardening Recommendations

Rather than presenting ambiguous, qualitative assessments, the system computes a normalized 0–100 risk score based on an objective, weighted mathematical formula. Baseline protocol weights are combined with additive vulnerability penalties for active risks (such as open networks, WEP, TKIP, WPS, and weak passwords) and clamped to the 0–100 range.

The numeric score maps directly to four intuitive risk classifications (`CRITICAL`, `HIGH`, `MEDIUM`, `LOW`). Furthermore, the engine dynamically generates targeted remediation steps, providing explicit instructions on disabling WPS, retiring legacy ciphers, setting high-entropy passphrases, and tunneling traffic through encrypted VPNs.

8.5 Secure Multi-User Portal with SHA-256 Authentication

The web dashboard is protected by an independent, multi-user authentication system. User registrations are validated against strict length constraints and case-insensitive duplicate checks to prevent account collision attacks.

To guarantee credential security, passwords are never stored in cleartext. Passwords are cryptographically transformed into 64-character hexadecimal digests using the industry-standard SHA-256 algorithm before storage in `users.json`, ensuring complete privacy and tamper resistance.

8.6 Interactive Cybersecurity Dashboard with Manual Auditing & Multi-Format Exports

The primary user interface is a dark-themed cybersecurity web dashboard featuring glowing risk badges, glassmorphism cards, and real-time metric counters. Users can review connection parameters, inspect vulnerability alerts, and delete obsolete records.

For advanced analysis, an interactive 'Manual Audit Modal' allows security practitioners to simulate hypothetical access point configurations without physically connecting to them. Furthermore, the dashboard provides one-click export controls, streaming formatted audit records in CSV and JSON formats for offline compliance documentation.

8.7 Dual Operational Modes (Web UI and CLI) and Comprehensive Test Suite

To support both interactive graphical environments and automated console deployments, the application provides dual interfaces: the Flask web server (`app.py`) and an independent terminal CLI (`main.py`). Both interfaces share identical analytical logic, guaranteeing uniform assessment results across environments.

Finally, the entire application is backed by an automated test suite comprising 29 unit and integration tests across `tests/`, ensuring flawless software stability, regression protection, and rigorous operational reliability.

CHAPTER 9 – FUTURE ENHANCEMENTS

9.1 Introduction

While the Wi-Fi Security Profiler achieves all core functional objectives, wireless network technologies and offensive threat vectors continue to evolve rapidly. Extending the system's capabilities will enhance its utility across enterprise networks, diverse operating systems, and automated defense pipelines.

This chapter outlines key architectural enhancements and research avenues planned for future development cycles, spanning cross-platform telemetry, machine learning anomaly detection, background daemon monitoring, packet capture analysis, and automated audit certificate generation.

9.2 Cross-Platform Wireless Telemetry Extraction

The current implementation extracts real-time hardware telemetry using Windows `netsh wlan`. A major planned enhancement is abstracting telemetry extraction across Unix-like operating systems. For Linux environments, the module will interface with NetworkManager's `nmcli` utility or direct `iwlib`/`wpa_cli` sockets. For macOS, telemetry will be extracted using the native `/System/Library/PrivateFrameworks/Apple80211.framework/Versions/Current/Resources/airport` tool.

Developing a unified cross-platform hardware interface layer will enable native execution on security-oriented Linux distributions (such as Kali Linux and Ubuntu) as well as macOS laptops without requiring code modifications.

9.3 Machine Learning-Based Rogue AP and Evil Twin Detection

A significant wireless threat is the deployment of Rogue Access Points and Evil Twin clones designed to hijack legitimate traffic. Future versions will incorporate machine learning anomaly detection algorithms (such as Isolation Forests and Support Vector Machines) to identify rogue access points dynamically.

The model will analyze multidimensional features, including RSSI signal variance, BSSID Organizationally Unique Identifier (OUI) vendor mismatches, beacon interval anomalies, and 802.11 supported rates. When an access point broadcasts a known SSID with anomalous radio parameters, the engine will flag it as a suspected clone.

9.4 Real-Time Continuous Background Monitoring and Audio/Desktop Alerts

Currently, telemetry extraction is triggered on demand by the user. A future release will introduce a lightweight background daemon process that monitors wireless interface state continuously at configurable intervals (e.g., every 30 seconds).

If the host machine connects to an unencrypted open network, experiences a sudden cryptographic cipher downgrade, or encounters an unexpected BSSID shift on a trusted SSID, the daemon will immediately trigger OS desktop push notifications and audible warnings, preventing inadvertent cleartext data transmission.

9.5 Packet Capture (PCAP) Analysis and Deauthentication Frame Detection

Future enhancements will integrate packet inspection capabilities using Python libraries such as Scapy or PyShark on compatible wireless adapters operating in monitor mode. This will enable the profiler to analyze raw 802.11 management frames over the air.

By monitoring 802.11 management frame traffic, the system will detect active deauthentication and disassociation frame floods (the signature of Wi-Fi jamming or Evil Twin handoff attacks) and alert users in real time when malicious frame injection occurs.

9.6 Automated PDF Audit Certificate Generation

To enhance professional auditing and organizational compliance, future versions will incorporate automated PDF report generation using libraries such as ReportLab. The system will compile evaluated profile metrics, detected vulnerabilities, hardening recommendations, and cryptographic timestamps into polished, multi-page security compliance certificates suitable for client delivery and formal administrative records.

CHAPTER 10 – CONCLUSION

10.1 Introduction

The proliferation of wireless local area networks has delivered unprecedented connectivity and operational mobility across modern society. However, the open, unguided nature of radio frequency transmission introduces significant cybersecurity risks that necessitate rigorous, continuous evaluation.

This concluding chapter synthesizes the project implementation accomplishments, reviews the technical effectiveness of Python as an engineering platform, reflects on key learning outcomes, evaluates project constraints, and presents final concluding observations.

10.2 Summary of Project Accomplishments

The Wi-Fi Security Profiler project successfully achieved all primary and secondary engineering objectives established at inception. The system delivers an automated, robust, and secure wireless auditing solution capable of interrogating host network adapters, extracting comprehensive connection telemetry, identifying captive portal splash pages, matching six distinct vulnerability patterns, and calculating objective 0–100 risk scores.

The application provides dual user interfaces—a modern cybersecurity web dashboard built with Flask and an independent terminal CLI—supported by multi-user SHA-256 cryptographic authentication, interactive manual auditing modals, and dynamic CSV and JSON report exports. The software is thoroughly verified by an automated test suite comprising 29 unit and integration tests, confirming 100% operational correctness.

10.3 Role of Python in Wi-Fi Security Auditing

The development lifecycle firmly demonstrated the exceptional suitability of Python for systems-level cybersecurity engineering. Python's rich standard library enabled seamless inter-process communication with host operating system utilities via `subprocess`, robust textual parsing via regular expressions (`re`), deterministic cryptographic hashing via `hashlib`, and structured persistence via `json` and `csv`—all without external driver dependencies.

Furthermore, combining Python's systems scripting capabilities with lightweight web frameworks like Flask facilitated the rapid delivery of a full-featured, visually compelling web application. Python's modular syntax allowed clean separation of concerns between hardware discovery, analytical scoring, user security, and interface presentation.

10.4 Results and Learning Outcomes

The research and implementation process provided invaluable practical insights across multiple cybersecurity domains. The project deepened theoretical and applied understanding of IEEE 802.11 standards, illuminating the cryptographic flaws of WEP and WPA-TKIP, the mechanics of the 4-way WPA2 handshake, the security innovations of WPA3-SAE, and the vulnerabilities inherent in WPS PIN pairing.

From a software engineering perspective, the project reinforced best practices in defensive programming, test-driven development, modular architecture, and session security. The successful construction of a transparent mathematical risk model demonstrated the importance of translating raw technical indicators into actionable, intuitive metrics for end-user protection.

10.5 Limitations and Overall Project Evaluation

While the Wi-Fi Security Profiler performs reliably across its designed scope, several technical limitations were identified during evaluation. Live hardware telemetry extraction is currently tailored to Microsoft Windows through `netsh wlan`, requiring future adaptation for Unix platforms. Additionally, the tool operates within standard user privileges and does not capture raw 802.11 radio frames in monitor mode.

Nonetheless, within its defined boundaries as a lightweight, non-intrusive defensive auditing utility, the system demonstrates outstanding efficiency, zero-configuration usability, and robust analytical accuracy.

10.6 Overall Project Significance and Conclusion

In conclusion, the Wi-Fi Security Profiler represents a meaningful contribution to endpoint wireless security awareness and automated vulnerability assessment. By empowering users to instantly recognize unencrypted links, captive portal risks, and obsolete ciphers, the system provides vital protection against eavesdropping, MITM interception, and credential compromise.

The project validates that robust, modular cybersecurity tools can be engineered effectively in Python, bridging the gap between low-level network protocols and intuitive user protection.

REFERENCES

[1] IEEE Computer Society. (2020). IEEE Standard for Information Technology - Telecommunications and Information Exchange Between Systems - Local and Metropolitan Area Networks - Specific Requirements - Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications. IEEE Std 802.11-2020.

[2] National Institute of Standards and Technology (NIST). (2012). Guidelines for Securing Wireless Local Area Networks (WLANs). NIST Special Publication (SP) 800-153. U.S. Department of Commerce.

[3] National Institute of Standards and Technology (NIST). (2007). Guide to Enterprise Telework and Remote Access Security. NIST Special Publication (SP) 800-46 Rev. 2. U.S. Department of Commerce.

[4] Wi-Fi Alliance. (2018). Wi-Fi Protected Access 3 (WPA3) Specification Version 1.0. Wi-Fi Alliance Technical Committee.

[5] Wi-Fi Alliance. (2004). Wi-Fi Protected Access 2 (WPA2) Specification. Wi-Fi Alliance Technical Committee.

[6] Fluhrer, S., Mantin, I., & Shamir, A. (2001). Weaknesses in the Key Scheduling Algorithm of RC4. Selected Areas in Cryptography (SAC 2001), Lecture Notes in Computer Science, Vol. 2259, pp. 1-24. Springer.

[7] Vanhoef, M., & Piessens, F. (2017). Key Reinstallation Attacks: Forcing Nonce Reuse in WPA2. Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS '17), pp. 1313-1328. ACM.

[8] Viehböck, S. (2011). Brute Forcing Wi-Fi Protected Setup: When Poor Design Meets Poor Implementation. Technical White Paper. Stefan Viehböck Security Research.

[9] RFC 8908. (2020). Captive-Portal Identification. Internet Engineering Task Force (IETF). Authors: T. Pauly, D. Thakore.

[10] RFC 8910. (2020). Captive-Portal Architecture. Internet Engineering Task Force (IETF). Authors: M. Kumari, E. Kline.

[11] Python Software Foundation. (2024). Python 3.12 Documentation: Interprocess Communication, Cryptographic Services, and Networking. Available at: https://docs.python.org/3/.

[12] Pallets Projects. (2024). Flask Documentation: Web Development, One Drop at a Time (Version 3.1.x). Available at: https://flask.palletsprojects.com/.